AgentForge

Dynamic AI Agent Builder Platform

Production-style architecture for agents that create and configure other agents

Designed for two developers | Azure deployment | BYOK model

A production-style Agentic AI platform where users describe tasks in plain English, and AgentForge creates specialized AI agents by designing goals, tools, workflows, prompts, memory, and validation rules.

Team 2 Developers	**Deployment** Azure Static Web Apps + Azure Container Apps	**LLM Cost** BYOK: users provide their own keys

1. Project Overview

AgentForge is a production-grade Agentic AI platform that enables users to create, configure, deploy, and manage specialized AI agents using natural language.

Instead of manually coding agents, users simply describe a task, and AgentForge automatically designs the agent workflow, selects appropriate tools, generates prompts, configures memory, and deploys the agent for execution.

The platform acts as an AI Agent Factory, where AI agents create and configure other AI agents for specific business, productivity, and automation tasks.

2. Problem Statement

Current AI platforms require users to manually perform several technical setup steps before an agent can be useful.

- Define agent roles
- Configure prompts
- Connect tools
- Manage memory
- Create workflows
- Deploy agents

This process requires technical expertise and significant setup time. AgentForge simplifies this by allowing users to create intelligent agents through natural language instructions.

3. Objective

The objective is to build a platform where users can describe a task in plain English and automatically receive a fully configured AI agent capable of performing that task.

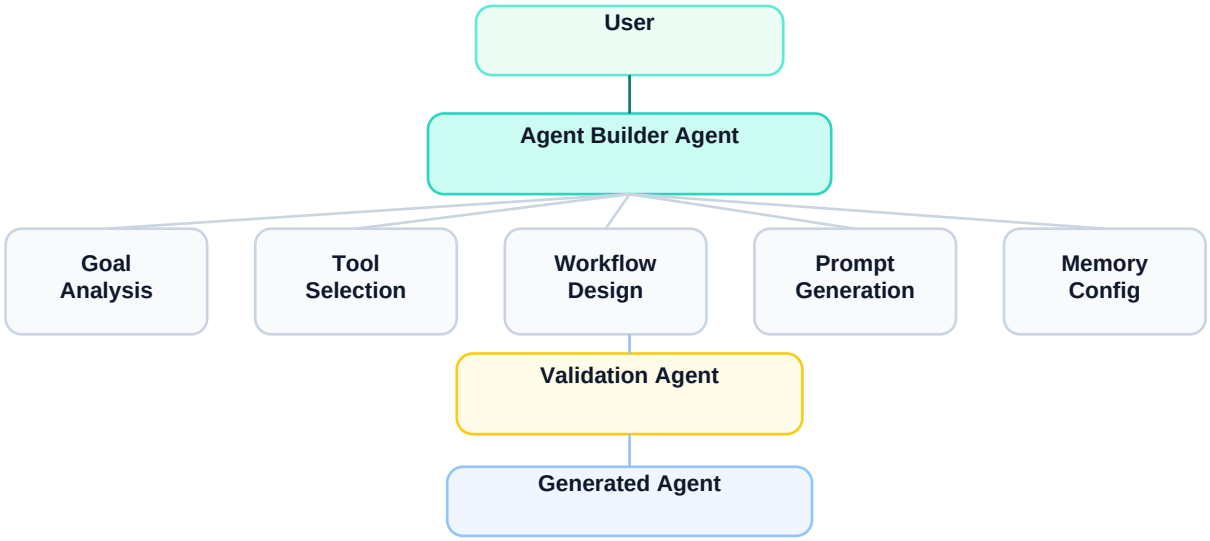
User Request

```
Create an agent that monitors LinkedIn for Data Engineer jobs and sends me daily summaries.
```

AgentForge Output

```
LinkedIn Job Tracker Agent Goal: Monitor Data Engineer jobs Tools: - Job Search Tool - Web Search Tool
- Email Notification Tool Memory: - Previously seen jobs - User preferences Schedule: - Daily at 9 AM
```

4. System Architecture



The Agent Builder Agent is the central orchestrator. It coordinates specialized internal agents that analyze the user goal, select tools, design workflows, create prompts, configure memory, and validate the generated agent before execution.

5. Core Components

Component	Purpose	Main Responsibilities
Agent Builder Agent	Central orchestration layer	Requirement analysis, agent blueprint generation, agent registration
Goal Analysis Agent	Understands user intent	Agent objective, expected output, frequency, required capabilities
Tool Selection Agent	Selects tools needed by the generated agent	Maps use cases to Gmail, search, AWS, notification, or web tools
Workflow Design Agent	Creates executable workflows	Builds workflow graphs using LangGraph
Prompt Generation Agent	Generates optimized prompts	Creates system prompts and role instructions
Memory Configuration Agent	Configures persistent memory	Stores previous executions, user preferences, historical results
Validation Agent	Checks agent readiness	Validates workflow correctness, tool availability, prompt quality, security constraints

6. Tool Selection Examples

Use Case	Tools
Email Assistant	Gmail API
Job Tracker	Search APIs
AWS Monitor	AWS Cost Explorer
Research Agent	Web Search

7. Workflow Design

Generated workflows are stored as executable graphs using LangGraph. A simple generated workflow may collect data, analyze it, summarize the result, and notify the user.

```
Collect Data | Analyze | Summarize | Notify User
```

8. Agent Lifecycle

```
User Request -> Analyze Goal -> Select Tools -> Create Workflow -> Generate Prompt -> Configure Memory
-> Validate Agent -> Deploy Agent -> Execute Tasks
```

9. Example Agents Generated by AgentForge

Generated Agent	Description
Job Tracking Agent	Tracks job postings based on user preferences and sends daily summaries.
Research Agent	Collects information from multiple sources and generates research reports.
Email Summary Agent	Summarizes inbox activity and highlights important emails.
Cloud Cost Monitoring Agent	Tracks cloud spending and generates alerts.
Market Monitoring Agent	Monitors stock prices, cryptocurrencies, or industry trends.

10. Technology Stack

Layer	Technology	Purpose
Frontend	React, TypeScript, Material UI, ReactFlow	User interface, forms, dashboards, workflow visualization
Backend	FastAPI, Python 3.12, Pydantic	API layer, validation, orchestration endpoints
Agent Framework	LangGraph	Multi-agent orchestration, stateful execution, dynamic workflows
Database	PostgreSQL on Neon	Users, agents, workflows, execution logs, memory
Vector Storage	pgvector	Agent memories, embeddings, semantic context, historical context
Authentication	Supabase Auth	Google login, GitHub login, email login

11. Database and Memory

PostgreSQL stores structured platform data, while pgvector stores semantic memories and embeddings. This avoids the need for a separate vector database in the first version.

- Users
- Agents
- Workflows
- Execution logs
- Memory
- Semantic embeddings
- Historical context

12. Azure Deployment Architecture

The frontend is hosted on Azure Static Web Apps. The FastAPI and LangGraph backend runs inside Azure Container Apps, which supports scale-to-zero behavior. The database uses Neon PostgreSQL with pgvector to keep early costs low while still maintaining a production-style architecture.

13. Cost Analysis

Service	Cost	Notes
Azure Static Web Apps	$0	Frontend hosting for React application
Azure Container Apps	$0-5/month	Backend container; scale-to-zero for low usage
Neon PostgreSQL	$0	Free PostgreSQL database for early-stage usage
Supabase Auth	$0	Authentication with Google, GitHub, and email login
Storage	$0	Initial storage can remain within free tiers

Total Infrastructure Cost: $0-5/month during early-stage usage.

LLM Costs: AgentForge follows a Bring Your Own Key model. Users provide their own OpenAI, Claude, or Gemini API keys, so platform LLM cost is $0. Users pay for their own model usage.

14. Key Features

Feature	Description
Dynamic Agent Generation	Agents are created automatically from natural language descriptions.
Multi-Agent Architecture	Specialized agents collaborate to create and validate new agents.
Persistent Memory	Agents retain context and learn from previous executions.
Tool Registry	Supports dynamic tool discovery and integration.
Workflow Generation	Creates executable workflows automatically.
BYOK Model Support	Supports multiple LLM providers without platform inference costs.
Scale-to-Zero Infrastructure	Backend runs only when users access the platform.

15. Expected Outcomes

- Reduce agent development time from hours to minutes.
- Enable non-technical users to create AI agents.
- Provide reusable, configurable, and scalable AI agents.
- Demonstrate advanced Agentic AI concepts such as multi-agent orchestration, memory management, workflow generation, and tool selection.

This project is realistic for two developers, highly relevant to modern AI engineering, and strong enough to serve as a flagship portfolio project for Agentic AI, AI Engineering, and Cloud Engineering roles.